Create and Update Relationships

Let's see now how to create data with relationships using these new **relationship attributes**. ✨

Create Instances with Fields

Let's check the old code we used to create some heroes and teams:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

There are several things to **notice** here.

First, we **create** some `Team` instance objects. We want to use the IDs of these teams when creating the `Hero` instances, in the `team_id` field.

But model instances **don't have an ID** generated by the database until we `add` and `commit` them to the **session**. Before that, they are just `None`, and we want to use the actual IDs.

So, we have to `add` them and `commit` the session first, before we start creating the `Hero` instances, to be able to **use their IDs**.

Then, we use those IDs when creating the `Hero` instances. We `add` the new heroes to the session, and then we `commit` them.

So, we are **committing twice**. And we have to remember to `add` some things first, and then `commit`, and do all that **in the right order**, otherwise we could end up using a `team.id` that is currently `None` because it hasn't been saved.

This is the first area where these **relationship attributes** can help. 🤓

Create Instances with Relationship Attributes

Now let's do all that, but this time using the new, shiny `Relationship` attributes:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Now we can create the `Team` instances and pass them directly to the new `team` argument when creating the `Hero` instances, as `team=team_preventers` instead of `team_id=team_preventers.id`.

And thanks to SQLAlchemy and how it works underneath, these teams don't even need to have an ID yet, but because we are assigning the whole object to each hero, those teams **will be automatically created** in the database, the automatic ID will be generated, and will be set in the `team_id` column for each of the corresponding hero rows.

In fact, now we don't even have to put the teams explicitly in the session with `session.add(team)`, because these `Team` instances are **already associated** with heroes that **we do** `add` to the session.

SQLAlchemy knows that it also has to include those teams in the next commit to be able to save the heroes correctly.

And then, as you can see, we only have to do one `commit()`.

Assign a Relationship

The same way we could assign an integer with a `team.id` to a `hero.team_id`, we can also assign the `Team` instance to the `hero.team`:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Create a Team with Heroes

Before, we created some `Team` instances and passed them in the `team=` argument when creating `Hero` instances.

We could also create the `Hero` instances first, and then pass them in the `heroes=` argument that takes a list, when creating a `Team` instance:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Here we create two heroes first, **Black Lion** and **Princess Sure-E**, and then we pass them in the `heroes` argument.

Notice that, the same as before, we only have to `add` the `Team` instance to the session, and because the heroes are connected to it, they will be automatically saved too when we `commit`.

Include Relationship Objects in the Many Side

We said before that this is a **many-to-one** relationship, because there can be **many** heroes that belong to **one** team.

We can also connect data with these relationship attributes on the **many** side.

As the attribute `team.heroes` behaves like a list, we can simply append to it.

Let's create some more heroes and add them to the `team_preventers.heroes` list attribute:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
```

```
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

The attribute `team_preventers.heroes` behaves like a list. But it's a special type of list, because when we modify it adding heroes to it, **SQLModel** (actually SQLAlchemy) **keeps track of the necessary changes** to be done in the database.

Then we `add()` the team to the session and `commit()` it.

And in the same way as before, we don't even have to `add()` the independent heroes to the session, because they are **connected to the team**.

Recap

We can use common Python objects and attributes to create and update data connections with these **relationship attributes**. 😎

Next we'll see how to use these relationship attributes to read connected data. 🤝